

MSDM 5001

An Introduction to Computational and Modelling Tools

Lecture 6: Shared-Memory Parallel Computing

Python 3.13t installation guide with uv

- Install uv following instructions on [uv](#)
- Create a virtual environment for Python 3.13t with
`uv venv <name> --python 3.13t`
- Activate the environment with
`source <name>/bin/activate`
- Type in
`python -c 'import sys; print(sys._is_gil_enabled())'`
- If “false” is printed, then GIL has been disabled

Shared-memory parallel computing

- Workers = threads in a process
 - Share the same main memory, but may have their own private “workspace”
- “Multithreading”
- c.f. “Distributed computing” or Python’s “multiprocessing”

Python's threading module

- Provides multi-threading functionalities
- Provides synchronization primitives we discussed
- Typical usage:

```
def worker(data):  
    # work on data, may involve synchronization  
  
threads = []  
  
# create and start threads  
for data in items():  
    # create threads  
    t = threading.Thread(target=worker, args=(data,))  
    threads.append(t)  
    t.start()  
  
# wait for threads to finish  
for t in threads:  
    t.join()
```

Dangers with sharing memory

- Consider the following program:

```
#include <iostream>
#include <thread>

int counter = 0;

void increment() {
    for (int i = 0; i < 1000000; i++) {
        counter++;
    }
}

int main() {
    std::thread t1(increment);
    std::thread t2(increment);

    t1.join();
    t2.join();

    std::cout << "Final counter value: " << counter << "\n";

    return 0;
}
```

Questions:

1. What would be the final counter value?
2. What if I write the same code in, say, Python 3.10?

Atomic operations

- Operations like ++ are NOT atomic
 - Load `counter` into register
 - Increment counter
 - Write `counter` back to memory
- Other threads may interfere in the middle of these operations!
 - Question: Who decides the ordering of threads?
- An example of “race conditions”
 - Result depends on (non-deterministic) ordering of thread executions

Strong vs weak memory models

- Recall CPU can execute instructions out-of-order provided the final result is the same
- What if we do this in a multi-core setting?
Initially both x and y are set to 0.
 - Thread 1: $x=1, r1=y$
 - Thread 2: $x=1, r2=x$

Both $r1$ and $r2$ may read 0 at the end!
- Strong vs weak memory model
 - Strong: Does not allow this kind of reordering (e.g. x86)
 - Weak: Allows more aggressive reordering to optimize for performance

Synchronization

- Mechanisms to coordinate different threads
 - Can help protect us from race conditions
 - We won't cover exact underlying implementations

```
#include <iostream>
#include <thread>

int counter = 0;

void increment() {
    for (int i = 0; i < 1000000; i++) {
        counter++;
    }
}

int main() {
    std::thread t1(increment);
    std::thread t2(increment);

    t1.join();
    t2.join();

    std::cout << "Final counter value: " << counter << "\n";

    return 0;
}
```

Can you think of a way to prevent having race condition in this code?

Busy waiting

- Main thread defines a shared integer `flag` to be 0

```
thread_id = get_thread_id()
while (flag != thread_id):
    continue
    # code that involves shared resources
flag += 1
```

- Obvious idea, but any issues?

Locks/mutexes

- Ensure only one thread can access a **critical section** of the code
- Two main operations:
 - Acquire/lock
 - Release/unlock
- If a thread does not have the lock when entering a critical section, it is **blocked**, i.e. put to sleep, and awoken only if the thread holding the mutex exits
- Typical usage

```
lock.acquire()  
# critical section that accesses shared resources  
lock.release()
```

An issue with thread ordering

- Consider a producer-consumer problem:
 - In a factory, type-A workers (producers) are responsible for loading materials into a storage box (a “buffer”), and type-B workers (consumers) are responsible for taking materials from the box and processing it
 - The box has finite capacity
 - Do you see any potential problems with this set up?
 - Buffer overflow
 - Empty buffer
 - Simultaneous access of buffer
- } About thread-ordering! Mutex will not help

Semaphore

- A fancy counter with two atomic operations
 - `acquire()/wait()`: decrement counter, block if below zero
 - `release()/signal()`: increment counter, wake up waiting threads
- Any thread can signal, any thread can wait



Using semaphores to handle producer- consumer pattern

```
# Shared buffer (simple list)
buffer = []
BUFFER_SIZE = 5

# Semaphores
empty_slots = threading.Semaphore(BUFFER_SIZE) # Tracks empty slots
filled_slots = threading.Semaphore(0)          # Tracks filled slots
buffer_lock = threading.Lock()                 # Protects buffer access
```

```
def producer(producer_id):
    for i in range(5):
        item = f"Item-{producer_id}-{i}"

        empty_slots.acquire()    # Wait for an empty slot

        with buffer_lock:
            buffer.append(item)
            buffer_size = len(buffer)

        filled_slots.release()   # Signal that there's one more filled slot

        time.sleep(random.uniform(0.1, 0.5))
```

```
def consumer(consumer_id):
    for i in range(5):
        filled_slots.acquire()   # Wait for a filled slot

        with buffer_lock:
            item = buffer.pop(0)
            buffer_size = len(buffer)

        empty_slots.release()    # Signal that there's one more empty slot

        time.sleep(random.uniform(0.1, 0.5))
```

Mutex vs Semaphore

- Mutex
 - Ownership-based (“which thread has the mutex?”)
 - Only the owner can unlock
- Semaphore
 - Signal-based (“has proceed message been signaled?”)
 - Any thread can signal

Condition variables

- Sometimes we want to check for more complicated statements
 - The example before needs three semaphores, and is quite complex
- Condition variables allow users to wait on any condition
- Always come with a mutex (that locks the object on which the condition depends).
- Methods
 - `lock/unlock`: lock/unlock the associated mutex
 - `wait`: atomically release the lock and block the thread until notified
 - `notify`: wake up one waiting thread
 - `notify_all`: wake up all waiting threads

Using condition variables to handle producer-consumer pattern

why while?



```
# Condition variable and its associated lock
buffer_lock = threading.Lock()
buffer_condition = threading.Condition(buffer_lock)

buffer_lock = Lock()      # mutex for buffer access

not_full = Condition(mutex)
not_empty = Condition(mutex)

def producer(producer_id):
    for i in range(5):
        item = f"Item-{producer_id}-{i}"

        with buffer_condition:
            while len(buffer) >= BUFFER_SIZE: # Wait while buffer is full
                buffer_condition.wait()

            buffer.append(item)
            buffer_size = len(buffer)

            buffer_condition.notify() # Notify waiting consumers

            time.sleep(random.uniform(0.1, 0.5))

def consumer(consumer_id):
    for i in range(5):
        with buffer_condition:
            while len(buffer) == 0: # Wait while buffer is empty
                buffer_condition.wait()

            item = buffer.pop(0)

            buffer_condition.notify() # Notify waiting producers

            time.sleep(random.uniform(0.1, 0.5))
```


Condition variables vs Semaphores

- Semaphores
 - For counting-based conditions, no state to check
 - Easy cross-thread signaling (wait and acquire can be done by different threads)
- Condition variables
 - More general, still lock-based
 - Awkward cross-thread signaling

Barrier

- Suppose we want to time a multi-threaded system
 - Want all threads to start at the same time, and report time taken by the slowest thread
 - What if different threads start at different times??
- Barrier forbids threads to proceed until they have all arrived

Barrier

- Creating barriers: `barrier = Barrier(parties=n)`
- Waiting at a barrier: `barrier.wait()`
 - Block until `n` threads have called `wait()`

```
results = []
barrier = Barrier(parties=n)

def worker(barrier, results):
    barrier.wait()
    start = time.time()

    time.sleep(random.uniform(0.1, 0.3)) # simulate random work

    elapsed = time.time() - start

    results.append(elapsed)
```

Deadlocks

```
# Shared resources
resource_A, resource_B
lock_A, lock_B

thread_1():
    lock_A.acquire()
    some_computation() # only involves resource_A

    lock_B.acquire()
    some_other_computation() # only involves resource_B

    lock_B.release()
    lock_A.release()

thread_2():
    lock_B.acquire()
    some_other_computation() # only involves resource_B

    lock_A.acquire()
    some_computation() # only involves resource_A

    lock_A.release()
    lock_B.release()
```

What happens when I start the two threads?

How to avoid deadlocks?

- Simplest: establish a lock ordering, respected by all threads
- Other algorithms (out of scope of this course):
 - Dekker's algorithm
 - Banker's algorithm
 - ...

concurrent.futures()

- Higher-level thread management (can also be used for processes)
 - Uses a pool of threads created upfront
 - Threads can be reused
- Use threading if you want fine-grained control, otherwise use concurrent.futures
- A typical usage pattern

```
from concurrent.futures import ThreadPoolExecutor

def worker(data):
    result = process(data)
    return result

with ThreadPoolExecutor(max_workers=num_threads) as executor:
    # data_chunks is a list of chunks of data
    results = list(executor.map(worker, data_chunks))
```

Exercise 1: Parallel Sum

- Goal: Implement a version of parallel sum algorithm:
 - Different threads get their own portion of an array, and compute their partial sums
 - A master thread (say thread 0) then combines the partial sums.
 - DO NOT COMPUTE THE FINAL SUM IN THE MAIN THREAD
- Complete the code in `Exercise_1_Blank.py` on Canvas

Exercise 2: Prime Counting

- Goal: Count how many numbers in the first n natural numbers are prime.
- You need to
 - Use a global counter
 - Write a worker function that determines if a given number is prime
 - Create a fixed number of threads (say 8) to carry out the work
 - Divide up the work using a static schedule. Try out different scheduling schemes.
 - Now use `concurrent.futures` instead.

Resources

- Chapter 4 from “An Introduction to Parallel Programming” by Peter Pacheco
(It is in C++, but most of the ideas are directly transferable)